

# B2B SaaS Platform Testing – Multi-Platform Automation

## Overview

You're joining the QA team at "WorkFlow Pro" - a B2B project management SaaS platform. The platform serves multiple clients (multi-tenant), works across web browsers and mobile devices, and integrates with various third-party services.

## Time Allocation

- **Take-Home Portion:** 90 minutes maximum
- **Live Discussion:** 30-45 minutes (scheduled separately)

## Part 1: Debugging Flaky Test Code (30 minutes)

A previous intern wrote this Playwright test for user login functionality. **The test is flaky** - sometimes passes, sometimes fails in CI/CD pipeline.

```
import pytest
from playwright.sync_api import sync_playwright

def test_user_login():
    with sync_playwright() as p:
        browser = p.chromium.launch()
        page = browser.new_page()

        # Navigate to login page
        page.goto("https://app.workflowpro.com/login")

        # Fill login form
        page.fill("#email", "admin@company1.com")
        page.fill("#password", "password123")
        page.click("#login-btn")

        # Verify successful login
        assert page.url == "https://app.workflowpro.com/dashboard"
        assert page.locator(".welcome-message").is_visible()

        browser.close()

def test_multi_tenant_access():
    with sync_playwright() as p:
        browser = p.chromium.launch()
        page = browser.new_page()

        page.goto("https://app.workflowpro.com/login")
        page.fill("#email", "user@company2.com")
        page.fill("#password", "password123")
        page.click("#login-btn")

        # User should only see Company2 data
        projects = page.locator(".project-card").all()
        for project in projects:
            assert "Company2" in project.text_content()
```

```
browser.close()
```

### Your Tasks:

1. **Identify Flakiness Issues:** List all problems that could cause intermittent failures
2. **Explain Root Causes:** Why do these issues happen in CI/CD vs local environment?
3. **Provide Fixes:** Write the corrected version with proper waits, error handling, and reliability improvements

### Additional Context (you may need to ask for more):

- App uses dynamic loading for dashboard elements
- Login process includes 2FA for some users
- Different tenants have different loading times
- CI runs on different browsers and screen sizes

## Solution

### 1. Flakiness Issues Identified

- **No waits before interacting with the form:** `page.fill()` and `page.click()` run right after `goto()` with no guarantee the login form has finished rendering, so on a slow load the selectors simply aren't there yet.
- **Hard-coded URL assertion:** `assert page.url == "...dashboard"` checks the URL exactly once. If the redirect after login is still in progress, this fails even though login actually succeeded a moment later.
- **`is_visible()` does not wait:** Unlike `expect(locator).to_be_visible()`, calling `.is_visible()` directly returns the state at that instant. Since the dashboard loads asynchronously, the welcome message may not be in the DOM yet.
- **2FA is not handled at all:** The context says some accounts require 2FA. The script assumes login always drops straight onto the dashboard, so any 2FA-enabled account will get stuck after clicking login.
- **Project list read before it finishes loading:** `page.locator('.project-card').all()` grabs whatever cards exist right now. Since different tenants have different loading times, this can read an empty or partial list and produce a false pass/fail.
- **No custom timeouts:** Default Playwright timeouts may not be enough for a slower CI runner, and there's no way to tell from the error whether it was a real bug or just "too slow".
- **Hard-coded credentials:** Login/password are written directly in the test file, which is a security concern and also means the whole suite breaks if that one account is disabled or its password is rotated.
- **No try/finally around `browser.close()`:** If any assertion fails, the browser process is never closed. On CI, repeated leaks like this eat up memory and cause unrelated tests to fail later in the run.
- **No viewport configuration:** CI runs across different screen sizes, but the test never sets one. At a smaller default resolution, elements can end up hidden behind a responsive/mobile layout.

- **Selector strategy is fragile:** IDs and class names (#login-btn, .welcome-message) can change any time the front-end team restyles the page, which breaks tests that have nothing to do with the actual bug.

## 2. Root Causes (Local vs CI/CD)

- **Network latency:** CI runners usually don't have the same fast connection or warm caches a developer's laptop has, so page loads and API calls genuinely take longer.
- **Machine performance:** CI containers are often shared and resource-constrained, so rendering and JS execution are slower - the gap between "click" and "element ready" is bigger than it looks locally.
- **Asynchronous loading / race conditions:** The dashboard and project cards load via async calls. Locally this might resolve fast enough to look reliable; on CI the assertion can run before the data has actually rendered.
- **Different browsers:** CI pipelines typically test Chromium, Firefox and WebKit. Small differences in load timing and rendering between engines make timing-sensitive tests behave differently across browsers.
- **Different screen resolutions:** CI containers often default to a different viewport than a developer's monitor, which can change what's visible without an explicit wait or resize.
- **Environment/data differences:** 2FA settings, seed data, and feature flags can differ between the account used locally and the one used in CI, so the same script behaves differently for reasons unrelated to the code.
- **Resource cleanup:** Locally you usually run one test at a time with plenty of headroom. On CI, leaked browsers from a missing try/finally accumulate across the whole suite and start starving later tests.

## 3. Corrected Test Code

### Assumptions made (since some information isn't in the prompt):

- Test accounts used in automation are assumed to support a test-only 2FA bypass or a static OTP (real SMS/email 2FA isn't reliably automatable). This needs to be confirmed with the dev team.
- data-testid attributes may not exist yet on every element, so the code falls back to the original CSS selectors where needed.
- Credentials are pulled from environment variables instead of being hard-coded.

```
import os
import re
import pytest
from playwright.sync_api import sync_playwright, expect

BASE_URL = os.getenv("APP_BASE_URL", "https://app.workflowpro.com")
DEFAULT_TIMEOUT = 15000 # ms - generous enough to absorb CI latency

def login(page, email: str, password: str):
    """
    Reusable login helper. Waits for real signals (URL change,
    key element) instead of asserting immediately after a click,
    and optionally handles a 2FA prompt if one appears.
    """
    page.goto(f"{BASE_URL}/login", wait_until="domcontentloaded")
```

```

email_input = page.locator("#email")
password_input = page.locator("#password")
login_button = page.get_by_role("button", name=re.compile("log ?in", re.I))

expect(email_input).to_be_visible(timeout=DEFAULT_TIMEOUT)
email_input.fill(email)
password_input.fill(password)
login_button.click()

# --- Optional 2FA step ---
otp_input = page.locator('[data-testid="otp-input"]')
try:
    otp_input.wait_for(state="visible", timeout=5000)
    test_otp = os.getenv("TEST_OTP_CODE", "000000")
    otp_input.fill(test_otp)
    page.get_by_role("button", name=re.compile("verify|submit", re.I)).click()
except Exception:
    pass # No 2FA prompt for this account - continue normally

# Wait for the actual navigation instead of reading page.url once
page.wait_for_url(f"{BASE_URL}/dashboard", timeout=DEFAULT_TIMEOUT)

@pytest.fixture
def browser_page():
    """Provides a fresh page per test and guarantees cleanup."""
    with sync_playwright() as p:
        browser = p.chromium.launch()
        context = browser.new_context(viewport={"width": 1366, "height": 768})
        page = context.new_page()
        page.set_default_timeout(DEFAULT_TIMEOUT)
        try:
            yield page
        finally:
            context.close()
            browser.close()

def test_user_login(browser_page):
    page = browser_page
    login(
        page,
        os.getenv("TEST_USER_EMAIL", "admin@company1.com"),
        os.getenv("TEST_USER_PASSWORD", "password123"),
    )

    # expect() auto-retries until the condition is true or times out
    expect(page).to_have_url(f"{BASE_URL}/dashboard", timeout=DEFAULT_TIMEOUT)

    welcome_message = page.locator(".welcome-message")
    expect(welcome_message).to_be_visible(timeout=DEFAULT_TIMEOUT)

def test_multi_tenant_access(browser_page):
    page = browser_page
    login(

```

```

    page,
    os.getenv("TENANT2_USER_EMAIL", "user@company2.com"),
    os.getenv("TENANT2_USER_PASSWORD", "password123"),
)

project_cards = page.locator(".project-card")

# Wait for the list to actually render before reading it,
# since different tenants load at different speeds
expect(project_cards.first().to_be_visible(timeout=DEFAULT_TIMEOUT))

count = project_cards.count()
assert count > 0, "Expected at least one project card for the Company2 user"

for i in range(count):
    card_text = project_cards.nth(i).text_content()
    assert "Company2" in card_text, f"Unexpected tenant data leaked: {card_text}"

```

### Key improvements explained

- **expect() instead of raw assert:** auto-retries against the dynamic dashboard/project list instead of checking once, which removes most of the race-condition failures.
- **wait\_for\_url() after login:** waits for the real navigation event rather than reading page.url immediately after the click.
- **2FA handled with a bounded, non-blocking check:** if no OTP field shows up within a short timeout, the test just continues, so it still works for accounts without 2FA.
- **try/finally fixture:** guarantees the browser closes even when an assertion fails, so failures don't leak resources for later tests.
- **Fixed viewport:** removes the "different screen sizes" variable so layout-related failures are actually about the app, not the runner's default window.
- **Credentials via environment variables:** keeps secrets out of source control and makes it easy to swap accounts per environment.
- **Shared login() helper:** removes duplication between the two tests and means any future reliability fix only needs to happen in one place.

### Additional improvements for a real framework

- **Page Object Model:** LoginPage / DashboardPage classes so tests read like business steps instead of raw locator calls.
- **Secure credential storage:** CI secrets manager (GitHub Actions secrets, Vault) instead of a plain .env checked into the repo.
- **Retry on failure:** pytest-rerunfailures or Playwright's built-in retries to absorb genuine one-off CI blips.
- **Traces/screenshots on failure:** context.tracing.start()/stop() plus screenshot-on-failure so CI failures are debuggable without reproducing locally.
- **Parallel execution:** pytest-xdist, with each test fully isolated (its own context/tenant data) so parallel runs don't interfere with each other.
- **Cross-browser matrix:** parametrize over chromium/firefox/webkit and a couple of viewport presets.

## Part 2: Test Framework Design (25 minutes)

Design a test automation framework structure for this B2B SaaS platform. **Note:** Requirements are intentionally incomplete.

### Given Requirements:

- Support web (Chrome, Firefox, Safari) and mobile (iOS, Android) testing
- Handle multiple tenant environments (company1.workflowpro.com, company2.workflowpro.com)
- Test different user roles (Admin, Manager, Employee) with varying permissions
- API testing for backend services
- Integration with BrowserStack for cross-platform testing
- CI/CD pipeline integration

### Your Tasks:

4. **Framework Structure:** Design folder structure, base classes, and utilities
5. **Configuration Management:** How to handle multiple environments, browsers, and test data
6. **Identify Missing Requirements:** What questions would you ask about test data management, reporting, parallel execution, etc.?

*Format: Use text/pseudocode/diagrams to explain your design*

## Solution

### 1. Folder Structure

```
workflowpro-automation/
├── config/
│   ├── environments/
│   │   ├── company1.yaml    # base_url, tenant_id, default users
│   │   ├── company2.yaml
│   │   └── staging.yaml
│   └── browserstack.yaml    # capabilities for web + mobile devices
├── src/
│   ├── pages/               # Page Object Model
│   │   ├── base_page.py
│   │   ├── login_page.py
│   │   ├── dashboard_page.py
│   │   └── project_page.py
│   ├── api/
│   │   ├── api_client.py    # thin wrapper around requests/httpx
│   │   └── endpoints/
│   │       └── projects.py
│   ├── data/
│   │   ├── factories.py     # builds test data (projects, users)
│   │   └── fixtures/*.json
│   └── utils/
│       └── auth.py          # token/session helpers per role
```

```

├── waits.py
├── logger.py
├── tests/
│   ├── ui/
│   │   ├── test_login.py
│   │   └── test_multi_tenant_access.py
│   ├── api/
│   │   └── test_projects_api.py
│   ├── mobile/
│   │   └── test_project_mobile.py
│   └── integration/
│       └── test_project_creation_flow.py
├── conftest.py                # shared fixtures (browser, api client, test data)
├── pytest.ini
└── .github/workflows/ci.yml

```

## 2. Base Classes (conceptual)

```

class BasePage:
    def __init__(self, page):
        self.page = page

    def open(self, path: str):
        self.page.goto(f"{self.base_url}{path}", wait_until="domcontentloaded")

    def wait_and_click(self, locator, timeout=15000):
        locator.wait_for(state="visible", timeout=timeout)
        locator.click()

class LoginPage(BasePage):
    def login(self, email, password):
        self.page.locator("#email").fill(email)
        self.page.locator("#password").fill(password)
        self.page.locator("#login-btn").click()

```

## 3. Configuration Management

- **Environments:** one YAML/JSON file per tenant/environment (base\_url, tenant\_id, seeded users). A `--env=company1` CLI flag or environment variable picks which file `conftest.py` loads.
- **Browsers:** `pytest.mark.parametrize` (or a fixture reading `BROWSER=chromium|firefox|webkit`) drives which Playwright browser type is launched; BrowserStack capabilities live in a separate config so local runs and BrowserStack runs share the same test code.
- **Test data:** a small factory module builds throwaway objects (e.g. unique project names using a timestamp/uuid) so tests don't collide with each other or with previous runs, and a teardown step deletes anything created via the API.
- **Secrets:** credentials, API tokens and BrowserStack keys are never committed - they're pulled from CI secrets / a local `.env` that's git-ignored.

## 4. Missing Requirements - Questions I Would Ask

- **Test data:** Is there a dedicated seeded test tenant, or do tests need to create/clean up their own data via the API on every run?

- Reporting: Is there an existing reporting tool in use (Allure, ReportPortal), or should the framework pick one from scratch?
- Parallel execution: Can tenants/test accounts be used concurrently without state clashing, or does each parallel worker need its own isolated tenant/account?
- BrowserStack budget: Is there a fixed monthly parallel-session limit that should shape how many browsers/devices run per CI trigger?
- Roles: What exact permission differences exist between Admin/Manager/Employee that tests need to assert on (visible menu items, allowed actions, etc.)?
- CI triggers: Should the full cross-browser/mobile matrix run on every pull request, or only a smoke subset, with the full matrix reserved for nightly/merge-to-main runs?
- Flaky test policy: Is there an existing retry/quarantine process for tests that fail intermittently in CI?

### Part 3: API + UI Integration Test (35 minutes)

Create a test that validates the complete flow of project creation through both API and UI layers.

#### Business Scenario:

7. : Create a project via API
8. : Verify project appears correctly in web UI
9. : Check project is accessible on mobile
10. : Validate proper tenant isolation (project only visible to correct company)

#### API Endpoint Information:

```
POST /api/v1/projects
Headers: Authorization: Bearer {token}, X-Tenant-ID: {company_id}
Body: {"name": "Test Project", "description": "...", "team_members": [...]}
Response: {"id": 123, "name": "Test Project", "status": "active"}
```

#### Your Tasks:

11. **Write Integration Test:** Combine pytest (API) + Playwright (UI) + BrowserStack mobile testing
12. **Handle Test Data:** Show how you'd manage test data across API/UI
13. **Cross-Platform Validation:** Ensure the test works on multiple browsers and mobile devices
14. **Tenant Isolation:** Verify security boundaries between companies

#### Expected Test Structure:

```
def test_project_creation_flow():
    # 1. API: Create project
    # 2. Web UI: Verify project display
    # 3. Mobile: Check mobile accessibility
    # 4. Security: Verify tenant isolation
    pass
```

#### Your Tasks:

15. **Implement the test:** using pytest + Playwright + BrowserStack concepts
16. **Handle edge cases:** (network failures, slow loading, mobile responsiveness)

17. **Explain approach:** with comments on your testing strategy

*Hints: You'll need to make assumptions about authentication, test data cleanup, and mobile testing setup.*

## Solution

### Assumptions made:

- An admin API token per tenant is available via environment variables (API\_TOKEN\_COMPANY1, API\_TOKEN\_COMPANY2).
- BrowserStack credentials are set as BROWSERSTACK\_USERNAME / BROWSERSTACK\_ACCESS\_KEY, and the mobile session connects through their WebDriver/Playwright hub URL.
- Every project created by a test is deleted in a teardown step (via a DELETE /api/v1/projects/{id} call, assumed to exist) so runs stay independent.

### Implementation

```
import os
import uuid
import requests
import pytest
from playwright.sync_api import sync_playwright, expect

API_BASE_URL = os.getenv("API_BASE_URL", "https://api.workflowpro.com/api/v1")
APP_BASE_URL = os.getenv("APP_BASE_URL", "https://app.workflowpro.com")

class ProjectsApiClient:
    """Small wrapper so tests don't repeat headers/retry logic."""

    def __init__(self, token: str, tenant_id: str):
        self.session = requests.Session()
        self.session.headers.update({
            "Authorization": f"Bearer {token}",
            "X-Tenant-ID": tenant_id,
        })

    def create_project(self, name: str, description: str, team_members=None):
        response = self.session.post(
            f"{API_BASE_URL}/projects",
            json={
                "name": name,
                "description": description,
                "team_members": team_members or [],
            },
            timeout=10,
        )
        response.raise_for_status()
        return response.json()

    def delete_project(self, project_id: int):
        # Best-effort cleanup - failures here shouldn't fail the test itself
        try:
            self.session.delete(f"{API_BASE_URL}/projects/{project_id}", timeout=10)
        except requests.RequestException:
```

```

        pass

@pytest.fixture
def company1_api():
    return ProjectsApiClient(
        token=os.getenv("API_TOKEN_COMPANY1"),
        tenant_id="company1",
    )

@pytest.fixture
def company2_api():
    return ProjectsApiClient(
        token=os.getenv("API_TOKEN_COMPANY2"),
        tenant_id="company2",
    )

def test_project_creation_flow(company1_api, company2_api):
    """
    End-to-end check: a project created via the API for Company1
    (1) shows up correctly in the Company1 web UI,
    (2) is reachable from a mobile session via BrowserStack, and
    (3) is NOT visible to a Company2 user (tenant isolation).
    """
    project_name = f"QA Automation Project {uuid.uuid4().hex[:8]}"
    project = None

    try:
        # ---- 1. API: create the project ----
        project = company1_api.create_project(
            name=project_name,
            description="Created by automated integration test",
            team_members=["qa.bot@company1.com"],
        )
        assert project["status"] == "active"
        project_id = project["id"]

        # ---- 2. Web UI: verify the project is visible ----
        with sync_playwright() as p:
            browser = p.chromium.launch()
            context = browser.new_context(viewport={"width": 1366, "height": 768})
            page = context.new_page()
            try:
                _login(page, os.getenv("COMPANY1_USER_EMAIL"),
os.getenv("COMPANY1_USER_PASSWORD"))
                project_card = page.locator(f'.project-card:has-text("{project_name}")')
                expect(project_card).to_be_visible(timeout=15000)
            finally:
                context.close()
                browser.close()

        # ---- 3. Mobile: verify via BrowserStack ----
        _verify_on_mobile(project_name)

        # ---- 4. Security: tenant isolation ----

```

```

        with sync_playwright() as p:
            browser = p.chromium.launch()
            context = browser.new_context(viewport={"width": 1366, "height": 768})
            page = context.new_page()
            try:
                _login(page, os.getenv("COMPANY2_USER_EMAIL"),
os.getenv("COMPANY2_USER_PASSWORD"))
                leaked_card = page.locator(f'.project-card:has-text("{project_name}")')
                expect(leaked_card).to_have_count(0)
            finally:
                context.close()
                browser.close()

    finally:
        # ---- Cleanup: always attempt to remove the created project ----
        if project:
            company1_api.delete_project(project["id"])

def _login(page, email, password):
    page.goto(f"{APP_BASE_URL}/login", wait_until="domcontentloaded")
    page.locator("#email").fill(email)
    page.locator("#password").fill(password)
    page.locator("#login-btn").click()
    page.wait_for_url(f"{APP_BASE_URL}/dashboard", timeout=15000)

def _verify_on_mobile(project_name: str):
    """
    Connects to a real device via BrowserStack App Automate / Playwright
    and confirms the project is reachable from the mobile view.
    """
    bs_username = os.getenv("BROWSERSTACK_USERNAME")
    bs_access_key = os.getenv("BROWSERSTACK_ACCESS_KEY")
    bs_ws_endpoint = (
        f"wss://cdp.browserstack.com/playwright?caps="
        f'{{"browser": "chrome", "device": "Samsung Galaxy S22", "os_version": "12.0", '
f'"browserstack.username": "{bs_username}", "browserstack.accessKey": "{bs_access_key}"}}'
    )
    with sync_playwright() as p:
        browser = p.chromium.connect(bs_ws_endpoint)
        page = browser.new_page()
        try:
            _login(page, os.getenv("COMPANY1_USER_EMAIL"),
os.getenv("COMPANY1_USER_PASSWORD"))
            project_card = page.locator(f'.project-card:has-text("{project_name}")')
            expect(project_card).to_be_visible(timeout=20000)
        finally:
            browser.close()

```

## Test Data Management (API / UI)

- **Unique names:** every project name includes a short uuid suffix so parallel runs never collide and assertions can search for an exact match instead of "first card in the list".
- **Created via API, verified via UI:** setup goes through the API (fast, reliable) and only the actual thing under test (UI display, mobile access, isolation) goes through the browser.

- **Guaranteed cleanup:** the try/finally block deletes the project after the test regardless of pass/fail, so the tenant environment doesn't accumulate test data over time.

### Cross-Platform Validation

- **Web:** the same \_login/UI-check logic can be parametrized over chromium, firefox, and webkit to cover the required browsers.
- **Mobile:** \_verify\_on\_mobile connects to BrowserStack's remote browser/device instead of a local one, so the same assertions run against a real iOS/Android device without changing the test logic.

### Tenant Isolation

The isolation check explicitly logs in as a Company2 user and asserts the Company1 project card has a count of zero, rather than only checking that Company1 can see it. A positive check alone would miss a real security bug (data leaking across tenants).

### Edge Cases Considered

- **Network failures on the API call:** response.raise\_for\_status() surfaces API errors immediately with a clear failure instead of continuing with a None project object.
- **Slow loading:** all UI checks use expect() with an explicit timeout instead of a one-time check, so slower tenants don't cause false failures.
- **Mobile responsiveness:** the mobile check re-uses the same locator/text match as the desktop check, so it also catches cases where the project renders but is hidden by a broken responsive layout.
- **Partial failure cleanup:** cleanup lives in a finally block at the top level so a failed UI or mobile assertion still triggers project deletion.

## Submission Instructions

18. : Create a document with your responses to all three parts
19. : Include reasoning for each technical decision
20. : List assumptions about missing requirements or unclear specifications
21. : Show test framework thinking - not just individual test cases
22. : Be prepared to discuss testing strategy and tool choices in live session

## Live Session Topics (Preview)

During our video call, we'll discuss:

- Your approach to debugging flaky tests and prevention strategies
- Framework design decisions and scalability considerations
- How you'd handle test data management in multi-tenant environments
- BrowserStack integration strategy and cost optimization
- CI/CD pipeline integration and parallel execution
- Monitoring and reporting for test automation

## Evaluation Criteria

**Technical Skills:**

- Understanding of test automation tools (Playwright, pytest, BrowserStack)
- Knowledge of web/mobile testing challenges
- API testing and integration concepts
- Framework design and maintainability thinking

**Testing Strategy:**

- Ability to identify and fix flaky tests
- Understanding of multi-tenant testing challenges
- Cross-platform testing approach
- Test data management strategies

**Problem-Solving:**

- Systematic debugging approach
- Recognition of missing requirements
- Consideration of real-world constraints (CI/CD, cost, maintenance)

**Communication:**

- Clear explanation of testing decisions
- Ability to ask clarifying questions about ambiguous requirements
- Understanding of business impact of testing choices

*Note: This assessment evaluates your current automation skills and learning potential. We value thoughtful approaches over perfect implementations - focus on demonstrating your testing mindset and technical reasoning.*